

Note that the responsibility of sending event notifications is coded within the controller and not in the domain object, since it is not the primary responsibility of the *AddService*.

## Serialisation

The *UserRecord* is a DTO that we developed in the past. In the above code, we sent the same object as the value in the Kafka message. However, Kafka does not really transport Java objects. It only accepts byte array to remain platform-neutral. We must have a serialiser to convert the *UserRecord* into a byte array so that Kafka transports it.

Here is the serialiser, which is self-explanatory.

```
package com.glarimy.ums.app;
import org.apache.kafka.common.serialization.Serializer;
import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
public class UserSerializer implements Serializer<UserRecord>
{
    @Override
    public byte[] serialize(String topic, UserRecord user) {
        ObjectMapper mapper = new ObjectMapper();
        try {
            return mapper.writeValueAsBytes(user);
        } catch (JsonProcessingException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

## Configuration

Though we have finished the coding part, it is not sufficient. The template still requires the location of the Kafka cluster and the serialisers to be used for converting the keys and values to byte arrays. The following additions to the *application.properties* file do that job for us:

```
spring.kafka.bootstrap-servers=kafka:9092
spring.kafka.producer.key-serializer=org.apache.kafka.common.
serialization.StringSerializer
spring.kafka.producer.value-serializer=com.glarimy.ums.app.
UserSerializer
```

Note that the above configuration claims that the Kafka broker is listening on port 9092 and is running on the machine with the DNS name *kafka*. So, it is our responsibility to set up Kafka to run accordingly.

## Building the Docker image

Once the above code and configuration are ready, we can build the service and create the Docker image. The *Dockerfile*

that was developed in the previous part is still valid.

```
FROM maven:3.5-jdk-8 AS build
COPY src /usr/glarimy/src
COPY pom.xml /usr/glarimy
RUN mvn -f /usr/glarimy/pom.xml clean package -DskipTests
EXPOSE 8080
ENTRYPOINT ["java","-jar","/usr/glarimy/target/ums-add-
service.jar"]
```

And the following command builds the Docker image, as in the past:

```
sudo docker build -t glarimy/ums-add-service .
```

## Deploying Kafka cluster and services

Kafka runs as a cluster of one to any number of brokers along with Zookeeper. In other words, to set up the Kafka cluster, we need to run the Zookeeper server and also at least one Kafka broker. Since we want everything to run on the Docker environment, we need the Docker images of Zookeeper and Kafka. The most popular choices are *bitnami/zookeeper:3.8* and *bitnami/kafka:3.1*.

Since both the images are required to always run together, we can use the *docker-compose* tool to simplify the deployment. Using *docker-compose*, we can deploy multiple services in one go on a dedicated network so that they can access each other using the published names and port numbers. We can also manage the dependencies among the services as well.

The *docker-compose* tool expects a manifest file named *docker-compose.yml* in which the required list of services, their dependencies, etc, are declared.

Let's first declare the zookeeper service:

```
zookeeper:
  image: docker.io/bitnami/zookeeper:3.8
  ports:
    - "2181:2181"
  volumes:
    - "zookeeper_data:/glarimy"
  environment:
    - ALLOW_ANONYMOUS_LOGIN=yes
  networks:
    - glarimy
```

This declaration tells the *docker-compose* tool to run a container named *zookeeper* from the specified image on the network named *glarimy*. Also, it tells that the port 2181 of *zookeeper* is to be mapped to the same port number on the network. It mounts a volume and sets up some configuration for the container via *environment*.